

TFS Framework

Tsallis Feature System · Parkinson Voice Analysis

v1.1 · Maio 2026 · Python 3.9+

Versão	1.1 (Maio 2026)
Linguagem	Python 3.9+
Dependências core	numpy · scipy
Dependências opcionais	librosa · torch · soundfile · pandas
Módulos	5 (tsallis_core · tfs_ab · tfs_cd · tfs_e_vae · tfs_simulator)
Features totais	55 (A:13 + B:24 + C:8 + D:12) + representações q-VAE
Benchmark alvo	Iyer et al. 2023 — CNN/AUC=0.97 sobre espectrogramas

Sobre: O TFS é um pipeline científico para extração de biomarcadores acústicos da doença de Parkinson usando a Entropia de Tsallis S_q como medida de complexidade central. Opera sobre fonação sustentada (vogal /a/) e gera 55 features interpretáveis processadas por um q-VAE para aprendizado de representação não-extensiva.

Índice

1. Visão Geral da Arquitetura
 2. Instalação e Dependências
 3. Módulo 1 — tsallis_core.py
 4. Módulo 2 — tfs_ab.py (Blocos A e B)
 5. Módulo 3 — tfs_cd.py (Blocos C e D)
 6. Módulo 4 — tfs_e_vae.py (q-VAE)
 7. Módulo 5 — tfs_simulator.py
 8. Pipeline Completo — Uso Rápido
 9. Dicionário Completo de Features
 10. Fundamentos Físicos e Hipóteses
 11. Notas de Implementação e Bugs Corrigidos
-

1. Visão Geral da Arquitetura

O TFS é organizado em cinco módulos com responsabilidades únicas que formam um pipeline hierárquico. A premissa central é que a voz na DP mudou de *regime dinâmico* — transitando de um oscilador caótico controlado para um sistema de menor grau de liberdade. A Entropia de Tsallis S_q captura esta transição por ser sensível a correlações de longo alcance e distribuições de cauda pesada, que a Entropia de Shannon ($q \rightarrow 1$) é incapaz de detectar.

Módulo	Arquivo	Responsabilidade	Output
M1	<code>tsallis_core.py</code>	Primitivas matemáticas: S_q , q-sweep, q-softmax, q-VAE loss, q-divergência	Funções
M2	<code>tfs_ab.py</code>	Bloco A: q-Sweep amplitude (13 feat.) + Bloco B: TES subbandas (24 feat.)	37 features
M3	<code>tfs_cd.py</code>	Bloco C: S_q de F0/perturbação (8 feat.) + Bloco D: q-Cronoentropograma (12 feat.)	20 features
M4	<code>tfs_e_vae.py</code>	q-VAE com ELBO Tsallis: aprendizado de representação não-extensiva	Espaço latente Z
M5	<code>tfs_simulator.py</code>	Simulador biofísico (GlottalSource + VocalTract + NoiseLayer) para validação	Áudio sintético

Fluxo: `.wav` → [M2: load+preprocess] → [M2: A+B] + [M3: C+D] → vetor 55 feat. → [M4: q-VAE] → Z → classificador.

2. Instalação e Dependências

2.1 Dependências Core (obrigatórias)

```
pip install numpy scipy
```

2.2 Dependências Opcionais

Pacote	Necessário para	Instalação
librosa	Carregamento .mp3/.ogg, YIN para F0 (M3), resampling de alta qualidade	<code>pip install librosa</code>
torch	TsallisVAE completo com GPU. Sem torch: NumpyQVAE (PCA+SVD) ativado automaticamente	<code>pip install torch</code>
soundfile	Escrita de .wav no simulador (M5). Fallback: .npy	<code>pip install soundfile</code>
pandas	<code>batch_extract_AB()</code> e <code>batch_extract_CD()</code> retornam DataFrame	<code>pip install pandas</code>
scikit-learn	Pipeline de classificação downstream (SVM, RF, XGBoost)	<code>pip install scikit-learn</code>

2.3 Estrutura de Arquivos

```
projeto/
■■■ tsallis_core.py # M1 – DEVE estar no mesmo dir ou no sys.path
■■■ tfs_ab.py # M2 – from tsallis_core import ...
■■■ tfs_cd.py # M3 – from tsallis_core import ...
■■■ tfs_e_vae.py # M4 – from tsallis_core import ...
■■■ tfs_simulator.py # M5 – independente
■■■ dados/
■■■ HC/ (arquivos .wav controle saudável)
■■■ DP/ (arquivos .wav Parkinson)
```

■■ **CRÍTICO:** todos os módulos importam `tsallis_core` com `'from tsallis_core import ...'`. O arquivo `tsallis_core.py` DEVE estar no diretório de trabalho ou em um caminho do `sys.path`.

3. Módulo 1 — tsallis_core.py (Núcleo Matemático)

Versão 1.1 — Núcleo com correções de bugs críticos

Contém todas as primitivas matemáticas do framework. Todos os outros módulos importam exclusivamente daqui. A versão 1.1 corrigiu três bugs identificados no self-test.

3.1 Definição Central

$S_q = (1 - \sum p_i^q) / (q - 1)$ para $q \neq 1$ (Entropia de Tsallis)

$S_1 = - \sum p_i \cdot \ln(p_i)$ limite de Shannon ($q \rightarrow 1$)

PDF estimada por histograma com critério de Freedman-Diaconis:

$n_{\text{bins}} \text{ automático} = \max(10, \text{ceil}((\text{max}-\text{min}) / (2 \cdot \text{IQR} \cdot N^{(-1/3)})))$

3.2 API Completa

Função	Parâmetros chave	Retorno
tsallis_entropy	signal, q=1.3, n_bins=-1	float $S_q \geq 0$
q_sweep	signal, q_values=Q_DEFAULT, n_bins=-1	array: perfil $S_q(q)$
profile_features	profile, q_values=None	dict: decay_ratio, area, slope_q1, curvature_int
dS_q_dq_at_q1	signal, dq=0.1	float: derivada dS_q/dq em $q=1$ (negativo)
q_softmax	z, q=1.5	array: distribuição esparsa ($q>1$)
q_divergence	P, Q, q=1.3	float: $D_q(P Q) \rightarrow \text{KL}$ para $q \rightarrow 1$
tsallis_loss_numpy	y_pred, y_true, q=1.3	float: Bregman-Tsallis loss
tsallis_entropy_series	series, q=1.3	float: alias para séries temporais
validate_signal	signal, min_samples=100, normalize=False	array: sinal sem NaN/Inf
check_q_value	q	None / ValueError se q fora de (0.01, 4.0)

3.3 Bugs Corrigidos v1.1

Bug	Problema	Correção
q_softmax	$0^{(\text{negativo})} = \text{inf}$ quando base atingia zero	$\max(\text{base}, 0.0)$ antes do expoente + np.errstate
tsallis_loss_numpy	$y_{\text{true}}=0, q>1 \rightarrow 0^{(\text{negativo})} = \text{inf}$	Reformulação Bregman-Tsallis (Amid et al., NeurIPS 2019)
q_star	S_q é monotonamente decrescente em q $\rightarrow q^*=q_{\text{min}}$ sempre, sem discriminação	Removida. Substituída por decay_ratio, curvature_int, slope_q1

4. Módulo 2 — tfs_ab.py (Blocos A e B)

37 features de amplitude e subbanda espectral

Implementa o primeiro estágio de extração de features. Cuida de toda a cadeia desde o carregamento do arquivo .wav/.mp3/.ogg até a geração de 37 features entrópicas divididas em dois blocos fisicamente motivados.

4.1 Pipeline Interno

```
load_audio(filepath) → (signal, fs) # wav/mp3/ogg → float64 normalizado
preprocess(signal, fs) → signal # trim silêncio (RMS -40dB) + BPF 80-3800 Hz
extract_block_A(signal) → dict 13 features # q-sweep sobre amplitude
extract_block_B(signal, fs) → dict 24 features # TES: Sq por subbanda espectral
extract_blocks_AB(filepath) → dict 37+ features # interface principal (aceita
filepath ou signal)
```

4.2 Bloco A — q-Sweep sobre Amplitude (13 features)

Calcula o perfil entrópico $S_q(q)$ da distribuição de amplitudes do sinal inteiro. **Atenção:** a amplitude NÃO é normalizada — a normalização destruiria a estrutura estatística que diferencia hipofonia da DP de voz saudável.

Feature	Descrição	Hipótese DP vs HC
A_Sq_0.3...A_Sq_2.5	S_q para 9 valores de q (perfil)	$S_q(q \leq 1)$ maior em DP — distribuição mais dispersa
A_decay_ratio *	$S_q(q_{\max})/S_q(q_{\min})$ — decaimento relativo	Menor em DP → caudas mais pesadas (q-Gaussiana com $q > 1$)
A_area *	integral trapezoidal do perfil	Menor em DP → redução da complexidade global
A_slope_q1 *	dS_q/dq em $q=1$ (diferença central)	$ \text{slope} $ maior em DP → maior sensibilidade ao regime
A_curvature_int	$\int d^2S/dq^2 dq$ — não-linearidade	Maior em DP → perfil mais curvado (não-Gaussianidade)

4.3 Bloco B — Espectro Entrópico de Tsallis / TES (24 features)

Calcula S_q dentro de 8 subbandas fisiológicas para $q \in \{0.7, 1.3, 2.0\} \rightarrow 24$ features.

Band a	Hz	Correlato vocal
B1	80–250	Frequência fundamental F0 — instabilidade glótica
B2	250–500	Primeiro formante F1 — imprecisão articulatória
B3	500–1000	Segundo formante F2 — centralização de vogais na DP
B4	1000–2000	Terceiro formante F3 — articulação supraglótica
B5	2000–3000	F4 / ruído glótico — turbulência (rouquidão)
B6	3000–4000	Turbulência fricativa supraglótica — soprosidade

B7	4000–6000	Presença articulatória — redução de alta freq. na DP
B8	6000–8000	Ruído estocástico residual (NaN se $fs < 12$ kHz)

■■ Nyquist: B7 e B8 exigem $fs \geq 12$ kHz. Para $fs = 8$ kHz (padrão), B8 retorna NaN com *UserWarning*. Use $fs = 16$ kHz para todas as 24 features do Bloco B.

4.4 Uso Rápido

```
from tfs_ab import extract_blocks_AB, batch_extract_AB

feats = extract_blocks_AB("paciente_001.wav") # dict com 37+ features
feats = extract_blocks_AB(signal=sig, fs=16000) # a partir de array

from tfs_ab import batch_extract_AB
df = batch_extract_AB(["hc1.wav", "dp1.wav"], labels=[0, 1]) # retorna DataFrame
```

5. Módulo 3 — tfs_cd.py (Blocos C e D)

20 features de dinâmica glótica e evolução temporal da entropia

Enquanto A e B operam no domínio de amplitude e frequência, C e D adicionam duas dimensões inéditas: a complexidade entrópica da dinâmica glótica ciclo-a-ciclo (C) e a evolução temporal da entropia ao longo da emissão (D — q-Cronoentropograma).

5.1 Bloco C — Tsallis sobre F0 e Perturbação (8 features)

Feature	Descrição	Significado Físico
C_Sq_F0_q{0.7, 1.3, 2.0}	S _q da série F0 ciclo-a-ciclo para 3 valores de q	Complexidade da dinâmica glótica; q=2.0 detecta saltos abruptos de F0
C_Sq_dF0_q{1.3, 2.0}	S _q das diferenças inter-ciclo ΔF0	Irregularidade do jitter em diferentes regimes de não-extensividade
C_q_fit_F0 *	q* por grid search de q-Gaussiana sobre série F0	q*>1.5 indica caudas pesadas na distribuição de F0 → tremor ou instabilidade severa
C_Sq_amp_cycle_q1.3	S _q das amplitudes pico-a-pico por ciclo glótico	Irregularidade de amplitude; complementa o shimmer clássico
C_Sq_shimmer_q1.3	S _q do shimmer: $ A_{i+1}-A_i /\text{mean}(A_i, A_{i+1})$	Baixo em HC (regular); alto em DP (irregularidade não-extensiva)

5.2 Bloco D — q-Cronoentropograma (12 features)

Inovação central do framework: computa $C_q(j) = S_q[p_{\{w,j\}}(x)]$ para cada janela temporal j (50 ms, hop 10 ms), gerando uma trajetória temporal da entropia. Extrai 6 estatísticas para $q \in \{1.3, 2.0\} = 12$ features.

Feature	Descrição	Hipótese H3
D_mean_q{1.3, 2.0}	Média temporal de C _q (j)	Menor em DP: entropia média mais baixa
D_std_q{1.3, 2.0}	Desvio padrão de C _q (j)	Maior em DP: flutuações temporais da complexidade
D_onset_q{1.3, 2.0}	Média nos primeiros 250 ms	Captura a capacidade de estabilizar a fonação no início
D_steady_q{1.3, 2.0}	Média na região estacionária central	Estado estacionário entrópico menor em DP
D_delta_q{1.3, 2.0} *	S _q (final) – S _q (onset): gradiente temporal	H3: ΔS _q > 0 em DP (fadiga vocal); ΔS _q ≈ 0 em HC (estabilidade)
D_tau_q{1.3, 2.0}	Posição relativa do pico entrópico (0–1)	τ < 0.5 em HC (pico no onset); τ > 0.5 em DP (instabilidade tardia)

5.3 Uso Rápido

```
from tfs_cd import extract_blocks_CD, batch_extract_CD

feats = extract_blocks_CD("paciente_001.wav")
```

```
print(f"D_delta_q1.3 = {feats['D_delta_q1.3']:.4f}") # positivo indica fadiga (DP?)  
df = batch_extract_CD(["hc1.wav", "dp1.wav"], labels=[0,1])
```

6. Módulo 4 — tfs_e_vae.py (q-VAE)

Variational Autoencoder com ELBO Tsallis

Implementa um VAE cuja função de custo (ELBO) é generalizada pela Entropia de Tsallis. O espaço latente é aprendido sob a hipótese explícita de não-extensividade — prior e posterior variacional são tratados como q-distribuições, não Gaussianas clássicas.

6.1 ELBO Tsallis

```
ELBO_q = L_recon_q +  $\beta$  · D_q(q(z|x) || p(z))

L_recon_q = E[ |x_recon - x|^((2-q) / (2-q)) ] (MSE generalizado via q-log)
D_q = q-divergência posterior variacional vs prior N(0,I)
 $\beta$  = peso regularização ( $\beta$ -VAE; padrão  $\beta=1.0$ )
q = parâmetro Tsallis (q=1.0 → VAE padrão)
```

6.2 Arquitetura (PyTorch)

Componente	Arquitetura	Justificativa
Encoder	<code>Linear(D,H)→LN→ELU→Linear(H,H/2)→LN→ELU→[μ, log σ^2]</code>	LayerNorm estável com batch pequeno (n~100); ELU evita dying neurons
Decoder	<code>Linear(Z,H/2)→LN→ELU→Linear(H/2,H)→LN→ELU→Linear(H,D)</code>	Espelhado ao encoder; sem ativação final (features z-scored externamente)
Otimizador	<code>Adam(lr=1e-3) + ReduceLROnPlateau(patience=10)</code>	Early stopping após 20 épocas sem melhora no ELBO
Normalização	z-score fitado em <code>fit()</code> , persistido para <code>encode()</code>	Prevent posterior collapse; NaN → 0.0 automaticamente

6.3 Uso Rápido

```
from tfs_e_vae import build_qvae, QVAETrainer
import numpy as np

X = np.load("tfs_features.npy") # (n_sujeitos, 55)
y = np.load("labels.npy") # {0=HC, 1=DP}

model = build_qvae(input_dim=55, latent_dim=8, q=1.3, beta=1.0)
trainer = QVAETrainer(model, lr=1e-3, batch_size=16)
trainer.fit(X, n_epochs=150, verbose=True)

Z = trainer.encode(X) # (n, 8) representações latentes
errors = trainer.reconstruction_error(X) # score de anomalia por sujeito
```

6.4 Fallback NumPy (sem PyTorch)

Sem PyTorch, `build_qvae()` retorna `NumpyQVAE` (SVD truncado + q-regularização analítica). Útil para prototipagem rápida. Aviso emitido automaticamente.

7. Módulo 5 — tfs_simulator.py (Simulador de Voz)

Modelo fonte-filtro biofísico com parametrização Tsallis

Implementa o modelo fonte-filtro clássico para gerar voz sintética HC e DP. O parâmetro `q_regime` é integrado diretamente na geração de jitter e shimmer, tornando o simulador fisicamente coerente com a teoria não-extensiva.

Classe	Modelo	Parâmetros HC → DP
GlottalSource	Pulso Rosenberg-C + jitter/shimmer q-Gaussiano + tremor senoidal 5 Hz	jitter: 0.25→2.0% shimmer: 1.5→8% tremor_amp: 0→0.08
VocalTractFilter	3 formantes Butterworth 2ª ordem em cascata (Klatt 1980 simplificado)	F1: 700→650 Hz F2: 1220→1100 Hz BW1: 80→150 Hz
NoiseLayer	HNR + ruído subglotal + aspiração (passa-alta) + micropausas Poisson	HNR: 25→12 dB aspiration: 0.05→0.25 pause_rate: 0→1/s
VoiceSimulator	Orquestra os 3 componentes. <code>q_regime</code> (1.0 HC → 1.5–2.0 DP) modula jitter/shimmer	severity ∈ [0,1] interpola linearmente HC → DP máximo

7.1 Uso Rápido

```
from tfs_simulator import VocalParams, VoiceSimulator, simulate_dataset,
batch_simulate

# Um arquivo HC
sim = VoiceSimulator(VocalParams.healthy(sex='M'), sr=16000, seed=42)
sim.save("hc_001.wav", duration=3.0)

# Dataset completo em memória (30 HC + 30 DP)
audios, labels, sr = simulate_dataset(n_hc=30, n_dp=30, duration=3.0, seed=42)

# Dataset em disco com metadata.json
result = batch_simulate(n_hc=50, n_dp=50, output_dir="simulated/",
severity_range=(0.3, 0.9), sex='mixed', verbose=True)
```

8. Pipeline Completo — Uso Rápido

Exemplo completo desde arquivos de áudio até classificação, passando por todos os módulos.

[illegible]

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, cross_val_score

pipe = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1, probability=True))
auc_tfs = cross_val_score(pipe, X, y, cv=StratifiedKFold(5),
    scoring='roc_auc').mean()
auc_vae = cross_val_score(pipe, Z, y, cv=StratifiedKFold(5),
    scoring='roc_auc').mean()

print(f"AUC (55 features TFS): {auc_tfs:.3f}")
print(f"AUC (espaço latente q-VAE): {auc_vae:.3f}")
print(f"Benchmark Iyer et al. CNN: 0.970")
```

■ *Para datasets pequenos ($n < 100$): use Leave-One-Out CV. Reporte AUROC com IC 95% via bootstrap ($n_{bootstrap}=2000$) para comparabilidade.*

9. Dicionário Completo de Features (55 features)

Tabela de referência. * = candidatas prioritárias ao ranking de importância.

#	Nome	Bloco	Descrição
1–9	A_Sq_0.3 ... A_Sq_2.5	A	S _q do perfil de amplitude para 9 valores de q em Q_DEFAULT
10*	A_decay_ratio	A	S _q (q _{max})/S _q (q _{min}) — razão de decaimento do perfil
11*	A_area	A	Integral trapezoidal $\int S_q dq$ — complexidade global
12*	A_slope_q1	A	dS _q /dq em q=1 — sensibilidade ao regime entrópico
13	A_curvature_int	A	$\int d^2S/dq^2 dq$ — não-linearidade do perfil
14–16	B_b1_q{0.7, 1.3, 2.0}	B	S _q na banda F0 (80–250 Hz)
17–19	B_b2_q{0.7, 1.3, 2.0}	B	S _q na banda F1 (250–500 Hz)
20–22	B_b3_q{0.7, 1.3, 2.0}	B	S _q na banda F2 (500–1000 Hz)
23–25	B_b4_q{0.7, 1.3, 2.0}	B	S _q na banda F3 (1000–2000 Hz)
26–28	B_b5_q{0.7, 1.3, 2.0}	B	S _q na banda F4/ruído glótico (2000–3000 Hz)
29–31	B_b6_q{0.7, 1.3, 2.0}	B	S _q turbulência fricativa (3000–4000 Hz)
32–34	B_b7_q{0.7, 1.3, 2.0}	B	S _q presença articulatória (4000–6000 Hz)
35–37	B_b8_q{0.7, 1.3, 2.0}	B	S _q ruído residual (6000–8000 Hz) — NaN para fs=8kHz
38–40	C_Sq_F0_q{0.7, 1.3, 2.0}	C	S _q da série F0 ciclo-a-ciclo
41–42	C_Sq_dF0_q{1.3, 2.0}	C	S _q das diferenças inter-ciclo $ \Delta F0 $
43*	C_q_fit_F0	C	q* por ajuste de q-Gaussiana à distribuição de F0
44	C_Sq_amp_cycle_q1.3	C	S _q das amplitudes pico-a-pico por ciclo glótico
45	C_Sq_shimmer_q1.3	C	S _q da série de shimmer ciclo-a-ciclo
46–47	D_mean_q{1.3, 2.0}	D	Média temporal do q-cronoentropograma C _q (j)
48–49	D_std_q{1.3, 2.0}	D	Desvio padrão temporal de C _q (j)
50–51	D_onset_q{1.3, 2.0}	D	Média entrópica no onset (0–250 ms)
52–53	D_steady_q{1.3, 2.0}	D	Média entrópica no estado estacionário
54–55*	D_delta_q{1.3, 2.0}	D	$\Delta S_q = S_q(\text{final}) - S_q(\text{onset})$: gradiente temporal
56–57	D_tau_q{1.3, 2.0}	D	Posição relativa do pico entrópico temporal (0–1)

* features marcadas são candidatas prioritárias para ranking de importância (maior Cohen's d esperado entre DP e HC).

10. Fundamentos Físicos e Hipóteses

Cada feature do TFS é ancorada em uma hipótese fisicamente motivada sobre como a disartrofia hipocinética da DP altera a estrutura estatística do sinal vocal.

Hipótese	Claim Físico	Features Testadas	Direção Esperada
H1a	Distribuição de amplitudes mais dispersa em DP (hipofonia + tremor)	$A_{Sq_q \leq 1}$	$\uparrow$ em DP
H1b	Distribuição de amplitudes com caudas mais pesadas em DP (q-Gaussiana com $q > 1$)	A_{decay_ratio}	$\downarrow$ em DP
H2a	Heterogeneidade espectral entrópica: F0 e formantes mais instáveis na DP	$B_{b1-b4_q2.0}$	$\uparrow$ B1; $\downarrow$ B2–B4
H2b	Aumento de ruído de alta frequência (soprosidade, aspiração) na DP	$B_{b5-b7_q1.3}$	$\uparrow$ em DP
H3a	Série de F0 com caudas mais pesadas em DP (instabilidade neuromotora glótica)	$C_{q_fit_F0}, C_{Sq_F0_q2.0}$	$q^* \uparrow, S_q \uparrow$
H3b	Fadiga vocal: entropia aumenta ao longo da emissão em DP, estável em HC	$D_{delta_q\{1.3, 2.0\}}$	$\Delta S_q > 0$ em DP
H3c	Instabilidade temporal tardia: pico entrópico ocorre depois em DP que em HC	$D_{tau_q\{1.3, 2.0\}}$	$\tau > 0.5$ em DP

10.1 Por que Tsallis e não Shannon?

A Entropia de Shannon assume **ergodicidade** e **correlações de curto alcance** — condições que o sistema vocal humano viola sistematicamente. A voz é um oscilador caótico com memória de longo alcance; quando a DP reduz os graus de liberdade deste sistema, ela produz distribuições com leis de potência (caudas pesadas) que somente a Entropia de Tsallis com $q > 1$ é capaz de detectar de forma sensível.

10.2 O Parâmetro q como Biomarcador

O q^* estimado pela feature $C_{q_fit_F0}$ tem interpretação física direta: ele é o índice de não-extensividade da distribuição de F0. Se $q^* \rightarrow 1$ (Gaussiana), o sistema glótico opera em regime extensivo normal. Se $q^* \rightarrow 2$ (Cauchy), o sistema perdeu correlações de longo alcance e exibe tremor e instabilidade de cauda pesada — sinal de DP avançada.

11. Notas de Implementação

■ *normalize=False em validate_signal*

Para o Bloco A, o sinal NÃO deve ser normalizado. A normalização altera a distribuição de amplitudes e invalida H1. Normalização é usada apenas nos Blocos B, C e D.

■ *Nyquist e Bloco B*

Para $fs=8\text{kHz}$, B8 retorna NaN automaticamente. Sempre verifique fs dos arquivos clínicos antes de usar todas as 24 features do Bloco B.

■ *YIN vs Autocorrelação (Bloco C)*

O extrator de F0 usa YIN (librosa) quando disponível. Para fonação sustentada estável, autocorrelação é igualmente precisa. Para sinais de alta variabilidade (DP severo), YIN é preferível.

■ *q-VAE com datasets pequenos*

Para $n < 50$ sujeitos, use `batch_size=8–16` e `latent_dim=4`. Com $n < 30$, considere NumpyQVAE + PCA como baseline interpretável.

■ *Imputação de NaN*

features NaN (Bloco B fora do Nyquist, ou sinal muito curto) devem ser imputadas por mediana da coluna antes de qualquer classificador. Use `sklearn.impute.SimpleImputer(strategy='median')`.

■ *q_regime no Simulador*

O simulador usa `q_regime` para amostrar jitter e shimmer de q-Gaussianas. Para testar o pipeline com dados sintéticos antes de dados clínicos, use `severity_range=(0.5, 0.8)` para simular DP moderada a severa.

■ *Reprodutibilidade*

Todos os módulos aceitam seed para reprodutibilidade. Para resultados publicáveis, fixe `numpy global seed + seed do VoiceSimulator`.

Referências Chave: Tsallis C. (1988) — Possible generalization of Boltzmann-Gibbs statistics, *J. Stat. Phys.* | Amid et al. (2019) — Robust Bi-Tempered Logistic Loss, *NeurIPS* | Iyer et al. (2023) — IPC2 Parkinson voice analysis (benchmark AUC=0.97) | Blondel et al. (2019) — Sparse transformers via q-softmax | Klatt D. (1980) — Software for a cascade/parallel formant synthesizer